Transactional Apps

With the ABAP RESTful application programming model (RAP), you can develop and extend transactional SAP Fiori apps.

About Transactional Apps

Qualities

- Develop stateless apps
- Use OData services to ensure the interoperability and scalability of your apps
- Develop metadata-driven UIs with UI annotations and SAP Fiori elements for a consistent user experience
- Create services that are optimized for SAP HANA to make full use of all advantages like buffering and performance optimizations
- Create lifecycle-stable apps and services that can be extended without modifications
- Document your development objects with knowledge transfer documents (KTD)
- Localize and internationalize your apps

Common Development Patterns

- List page and object page pattern: The main use case when creating SAP Fiori apps is to use the list report in conjunction with an object page. While the list report lets users filter, view, and work with items (objects) organized in list (table) format, object pages let users work with objects, providing functionality to view, edit, and create objects.
- Draft capabilities: A draft is an interim version of an instance that has not yet been explicitly saved as an active version. With RAP, you can easily enable draft capabilities with minimal effort.
- Treeview: Treeviews let you display hierarchical data on the UI.
- UI reuse services for for all common requirements like change documents or notes: You can use released APIs and UI components to integrate standard processes or standard functionality in your application, like change documents, without having to implement the logic yourself.

> **Note**
>
> To get an overview of all UI patterns, see SAP Fiori Elements Feature Showcase App for RAP .

Common Annotations

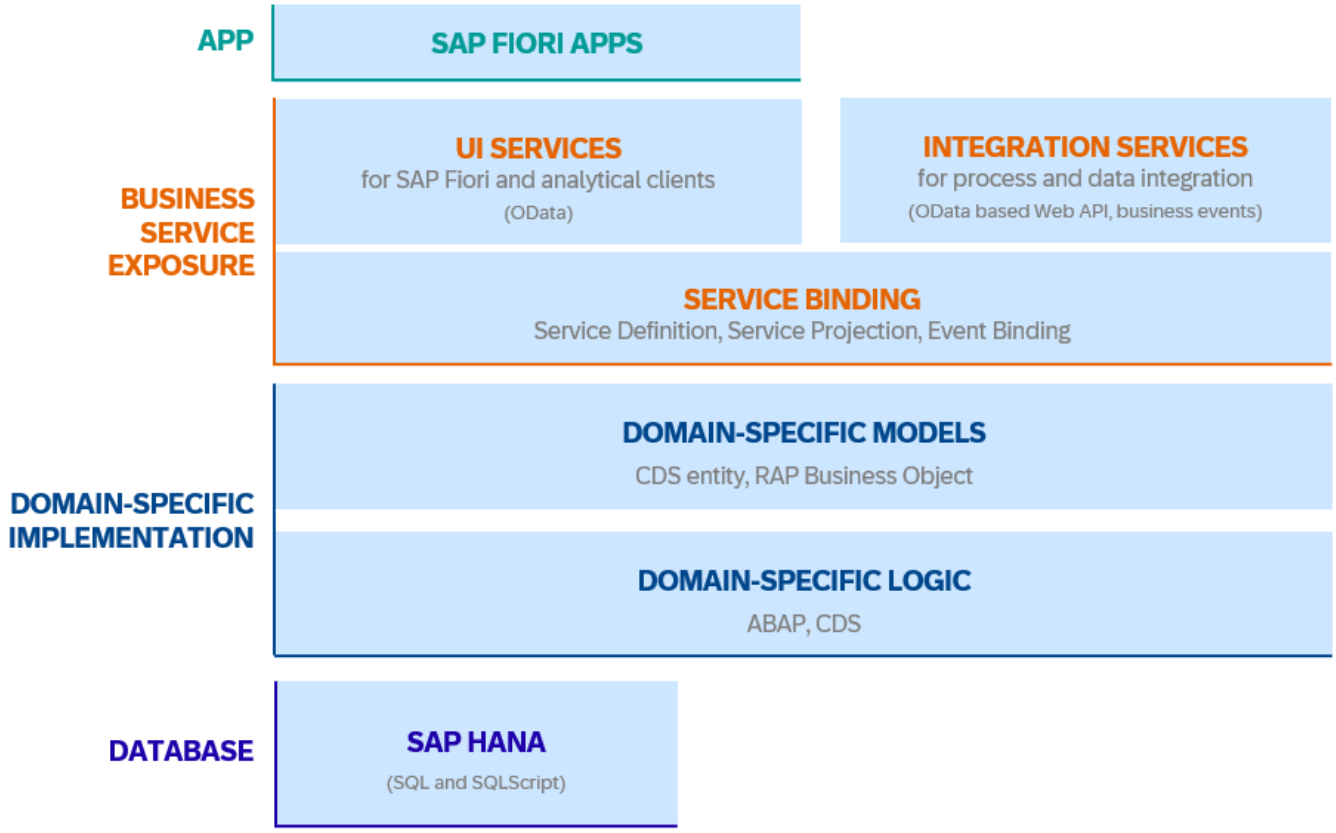
- UI Annotations: Define the syntax for a semantic views on business data by using specific patterns that are independent of used UI technologies.
- Consumption Annotations: Define a specific behavior that relates to the consumption of CDS content in domain-specific frameworks like the RAP query engine.
- Semantics Annotations: Allow the standardizing of semantics that only have an impact on the consumption side (such as currency code representation together with the amount).

Architecture

A business object (BO) is a common term to represent a real-world artifact in enterprise application development such as product or sales order. In general, a business object contains several nodes such as items and schedule lines and common transactional operations like creating, updating and deleting business data. The basis for a transactioal app is a RAP business object consisting of:

- a hierarchical data model with one root node and several child nodes.
- a behavior.
- the corresponding runtime implementation.

The different nodes of a RAP business object are modeled in a compositional hierarchy with ABAP CDS. The behavior is defined in a development object called the `behavior definition`, and the runtime behavior is implemented with ABAP and the `Entity Manipulation Language` in an ABAP behavior pool. With business object projections and interfaces you can define additional abstraction layers on top of the data model to adapt the underlying data model for different use cases.. Transactional services can be exposed as a UI service or a web API using OData or as business event using an event binding.



Develop Transactional Apps

Recommended Readings

- Data Modeling and Behavior
- Runtime Frameworks
- RAP BO Behavior
- Entity Manipulation Language (EML)
- Conceptual Background: Access Management (ABAP Cloud Guide)

Get Started

You can get started immediately with the `Create transactional app from database table` generator. All you need to to is create a database table and run the generator. The generator creates all relevant development objects, so you can get to know RAP business objects.

If you'd like to follow a step-by-step instruction to use the generator, refer to Generating a RAP Business Service with the Generate ABAP Repository Objects Wizards.

If you want to explore the available behavior in details, you read the chapter RAP BO Behavior in the documentation. If you prefer a hands-on approach you can import the RAP Flight reference scenario with sample code for all use cases described in the documentation.

Develop

Once you have created a basic transactional app, you can continue to finetune it by adding more complex behavior or by trying out other use cases like:

- Developing Transactional Apps with Multi-Inline-Edit Capabilities
- Developing a UI Service with Access to a Remote Service
- Developing Apps with Hierarchical Data Structures

Extend

Extending a SAP Fiori app delivered by SAP is also possible with RAP. You can find all the information about extensibility in RAP in the Extend chapter of the documentation.

If you're looking for potencial released APIs, you can refer to the SAP Business Accelerator Hub for an overview of released APIs for different SAP products.To find local released APIs in your system, see Finding Released APIs and Deprecated Objects.

Test

Unit and integration tests ensure that your app is always functional. You can find more information and samples in the Test chapter of the documentation.

Deploy

Development objects created for a transactional app rely on the respective lifecycle management of the respective products. For more information, see Lifecycle Management (ABAP Cloud guide).

More Information and Support

- SAP Community page for the ABAP RESTful application programming model : Here you can ask the RAP community for support in case you have questions.
- RAP documentation on the help portal: Contains all the information about RAP, its behavior, use cases and examples.
- ABAP resource library for transactional apps: Find an overview of all available resources for developing transactional apps.

Copyright | **Important Disclaimers and Legal Information**